



Healthcare Predictive Analytics Project

Supervised by Eng. Mahmoud Talaat

Team Members :

- Omar Hammed
- Mohammed Awad
- John Hosny
- Mahmoud Salah
- Mahmoud Nagaty
- Mohamed Aziz

Table of Contents

- 1-Introduction
- 2-Problem statement
- 3-Solution proposal
- 4-Methodology
- 5-Machine learning results
- 6-Conclusion

Introduction

Diabetes is one of the most widespread chronic diseases globally and poses a major health challenge, especially in developing countries. According to the International Diabetes Federation (IDF),

the number of people living with diabetes reached approximately 537 million in 2021, and this number is expected to rise to 643 million by 2030 if effective preventive measures are not taken. The danger of diabetes lies in the fact that a large portion of cases remain undiagnosed until complications arise, which may include heart disease, kidney failure, and vision loss.

Introduction

- In this project, we have developed a predictive model based on artificial intelligence techniques to determine whether a person is likely to have diabetes, using a set of life style related indicators. The goal of this model is to support early diagnosis efforts, improve quality of life, and reduce the burden on healthcare systems.

So bottom line of our research is,

What life-style choices are related to a higher risk of diabetes and can we help
Identifying patients without drawing a blood sample?

Problem Statement

Despite the growing number of people affected by diabetes, early and accurate diagnosis remains a challenge due to several real-world limitations:

- Individuals at risk often lack easy access to testing, especially in remote or underserved areas. Many would benefit from a reliable tool that allows them to self-assess at home without the need for immediate clinical visits.
- Junior or non-specialist doctors may struggle to confirm a diagnosis confidently in early or unclear cases, increasing the risk of misdiagnosis or delayed treatment.

Problem statement

- In emergency settings, nurses and paramedics frequently face time-sensitive situations where a quick and accurate assessment is needed, but full diagnostic tools are unavailable.
-
-
- These challenges highlight the need for an accessible, intelligent system that can assist in the early detection of diabetes, empowering both healthcare professionals and individuals.

Solution Proposal

- Predictive model for early diabetes Detection
- To address the challenges in diagnosing diabetes, we have developed an AI-based predictive model for early detection and self-assessment of diabetes. Self-Assessment Tool: The system allows individuals to check their risk of diabetes at home by entering key health indicators (such as age, weight, blood sugar levels). This helps users make informed decisions about seeking medical attention early. AI-Powered Model: The predictive model uses artificial intelligence to analyze data and predict the likelihood of diabetes with high accuracy. It can be used by individuals for self-checks and by healthcare providers to confirm diagnoses. Healthcare Support: This tool will assist doctors and emergency responders in diagnosing patients quickly, especially in emergency situations where timely and accurate decision-making is critical. This solution aims to provide easy access to diabetes diagnosis, improve early detection, and reduce the burden on healthcare systems

Methodology

- Data collection
- And Preprocessing
- Model Selection
- Training
- Evaluation
- Tuning
- Deployment
- And Maintenance

Data collection And Preprocessing

Source : https://www.cdc.gov/brfss/annual_data/annual_2015.html

The Behavioral Risk Factor Surveillance System (BFRSS)

The Behavioral Risk Factor Surveillance System (BRFSS) is a collaborative project between the Centers for Disease Control and Prevention (CDC) and all U.S. states, territories, and the District of Columbia.

Established in 1984, the BRFSS is the world's largest, continuously conducted telephone-based health survey system. Its primary objective is to collect uniform, state-specific data on preventive health practices and risk behaviors that are linked to chronic diseases, injuries, and preventable infectious diseases among the non-institutionalized adult population aged 18 years and older.

Data collection and Preprocessing

We have chosen the 2015 out of all years since it was the most documented and had an insightful codebook

Codebook: https://www.cdc.gov/brfss/annual_data/2015/pdf/codebook15_llcp.pdf

Explanation of Features:

the dataset contains nearly ~350 features, most of which are abundant and will not be considered in the research

- DIABETE4: Have you ever been told by a doctor that you have diabetes type 2? (boolean)
- SEXVAR: Gender (boolean)
- AGE5YR: Age group (num)
- EDUCA: Education (num)
- INCOME3: income salary (num)
- EXERANY2: did you exercise the past 30 days? (num)
- SMOKDAY2: are you a heavy smoker (boolean)
- _BMI5: what is your BMI (num)
- GENHLTH: rate your general health from 1 to 5 (num)
- PHYSHLTH: did you endure any health injury the past 30 days? (num)
- MENTHLTH : how many of the past 30 days would you say your mental health was bad (num)
- CHOLCHK3 : did you check your cholesterol level? (boolean), HIGHCHL: is your cholesterol high (boolean)
- CVDCHRD4: do you have a history with heart diseases? (boolean)
- BPHIGH5: did you check your blood level, if so was it high? (boolean)
- FRUITS, VEGIES, alcohol consumption, Past strokes, and DIFF walk are also accounted for totaling at 19 Features

Data collection and Preprocessing

- # check missing values per column
- # show only columns with missing values
- # % of missing values
- # check class distribution
- # Collapse target variable (0 vs 1+2)
- # One-hot encode Sex (0 = female, 1 = male in BRFSS)
- # BMI categories
- # General health score
- # (GenHlth: 1=Excellent, 5=Poor; MentHlth, PhysHlth are 1-30 days)
- # Diet quality score
- # Simply sum Fruits + Veggies (both are binary 0/1)
- # Age categories
- # Education mapping
- # Income mapping
- # One-hot encode all categorical features
- # Check class distribution again
- # create a cormobidtiy score
- # Bin MentHlth & PhysHlth (0=none, 1-10=mild, 11-20=moderate, 21-30=severe)
- # Drop low-variance columns
- #columns dropped

Data collection and Preprocessing

```
#253680 rows
# age category
# 1-Age 18 from to Age 24
# 2-Age 25 from to Age 29
# 3-Age 30 from to Age 34
# 4-Age 35 from to Age 39
# 5-Age 40 from to Age 44
# 6-Age 45 from to Age 49
# 7-Age 50 from to Age 54
# 8-Age 55 from to Age 59
# 9-Age 60 from to Age 64
# 10-Age 65 from to Age 69
# 11- Age 70 from to Age 74
# 12- age 75 to 79
# 13- Age 80 or older
# 14- Age 7 from to Age 9
```

```
# education level
# 1-Never attended school or only kindergarten
# 2-Grades 1 through 8 (Elementary)
# 3-Grades 9 through 11 (Some high school)
# 4-Grade 12 or GED (High school graduate)
# 5-College 1 year to 3 years (Some college or technical school)
# 6-College 4 years or more (College graduate)
# 7-Refused
```

```
# 1-Less than $10,000
# 2-Less than $15,000 ($10,000 to less than $15,000)
# 3-Less than $20,000 ($15,000 to less than $20,000)
# 4-Less than $25,000 ($20,000 to less than $25,000)
# 5-Less than $35,000 ($25,000 to less than $35,000)
# 6-Less than $50,000 ($35,000 to less than $50,000)
# 7-Less than $75,000 ($50,000 to less than $75,000)
# 8-$75,000 or more
# 77- Don't know/Not sure
# 99R -refused
# BLANK - Not asked or Missing
```

Data collection and Preprocessing

Handle missing values via knn imputation and collapsing variables for enhancing accuracy

```
missing_counts = df.isnull().sum()
print(missing_counts[missing_counts > 0])
missing_percent = df.isnull().mean() * 100
print(missing_percent[missing_percent > 0])
imputer = KNNImputer(n_neighbors=3)
df_filled = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
for col in df.columns:
    print(f"{col}: {df[col].unique()[:20]}")
```

[2]

```
df["Diabetes_binary"] = df["Diabetes_012"].apply(lambda x: 0 if x == 0 else 1)
df = pd.get_dummies(df, columns=["Sex"], prefix="Sex")
```

```
if "GenHlth" in df.columns:
    df["GenHlth_reversed"] = 6 - df["GenHlth"]
def bin_health(x):
    if x == 0:
        return 0 # none
    elif 1 <= x <= 10:
        return 1 # mild
    elif 11 <= x <= 20:
        return 2 # moderate
    else:
        return 3 # severe

if "MentHlth" in df.columns:
    df["MentHlth_binned"] = df["MentHlth"].apply(bin_health)
if "PhysHlth" in df.columns:
    df["PhysHlth_binned"] = df["PhysHlth"].apply(bin_health)
```

Data collection and Preprocessing

- Now binning variables and adding new features

```
df["GeneralHealthScore"] = df["GenHlth"] + df["MentHlth"] + df["DietQuality"] = df["Fruits"] + df["Veggies"]  
[6]
```

```
age_map = {  
    1: "18-24",  
    2: "25-29",  
    3: "30-34",  
    4: "35-39",  
    5: "40-44",  
    6: "45-49",  
    7: "50-54",  
    8: "55-59",  
    9: "60-64",  
    10: "65-69",  
    11: "70-74",  
    12: "75-79",  
    13: "80+",  
    14: "7-9"  
}  
df["Age_Category"] = df["Age"].map(age_map)  
[7]
```

```
edu_map = {  
    1: "Kindergarten",  
    2: "Elementary",  
    3: "SomeHighSchool",  
    4: "HighSchoolGrad",  
    5: "SomeCollege",  
    6: "CollegeGrad",  
    7: "Refused"  
}  
df["Education_Level"] = df["Education"].map(edu_map)  
[8]
```

```
categorical_cols = ["BMI_Category", "Age_Category", "Education_Level", "Income_Level"]  
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)  
[10]
```

```
1 def bmi_category(bmi):  
2     if bmi < 17:  
3         return "Underweight"  
4     elif 17 <= bmi <= 25:  
5         return "Normal"  
6     elif 26 <= bmi <= 33:  
7         return "Overweight"  
8     else:  
9         return "Obese"  
10  
11 df["BMI_Category"] = df["BMI"].apply(bmi_category)  
[5]
```

Data collection and preprocessing

Binary Target Counts:

Diabetes_binary

0 213703

1 39977

Name: count, dtype: int64

Binary Target Percentages:

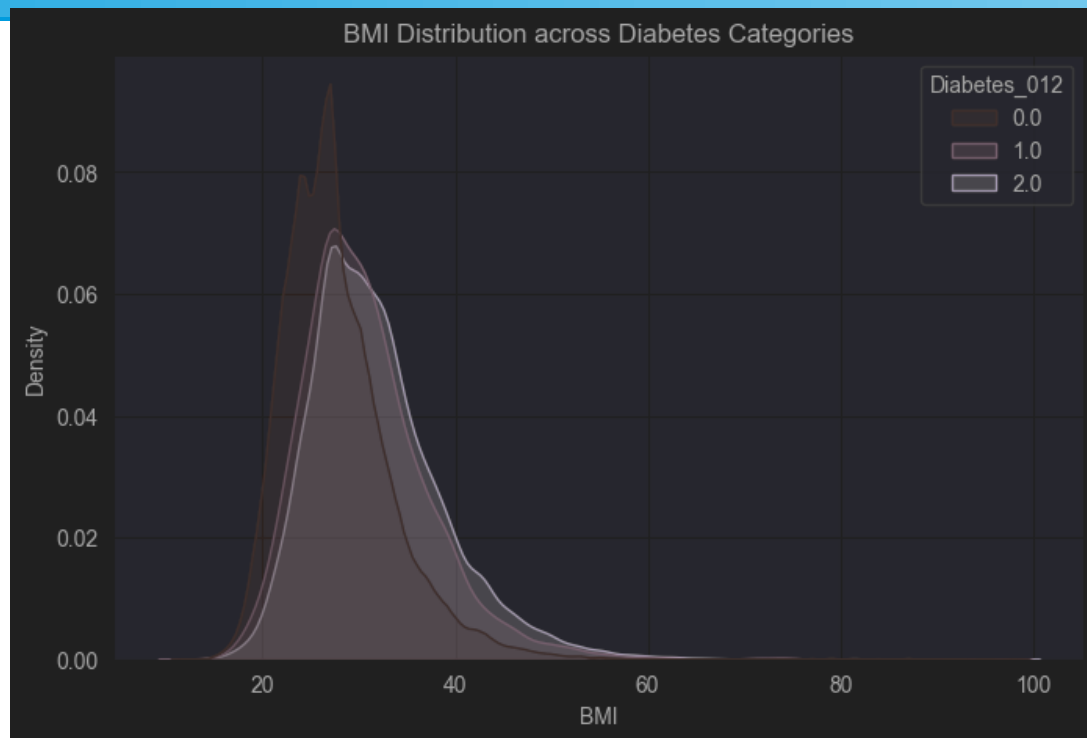
Diabetes_binary

0 84.24117

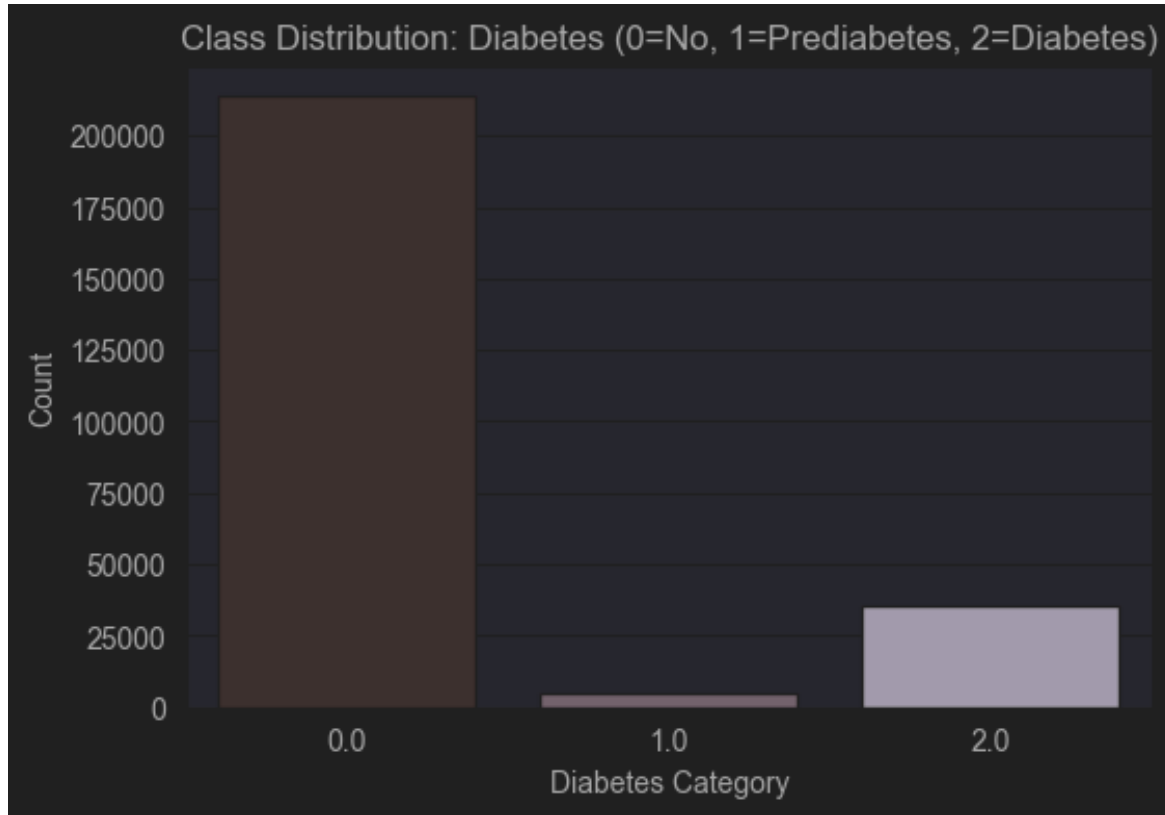
1 15.75883

Name: proportion, dtype: float64

Final dataset shape: (253680, 56)



Major class imbalance

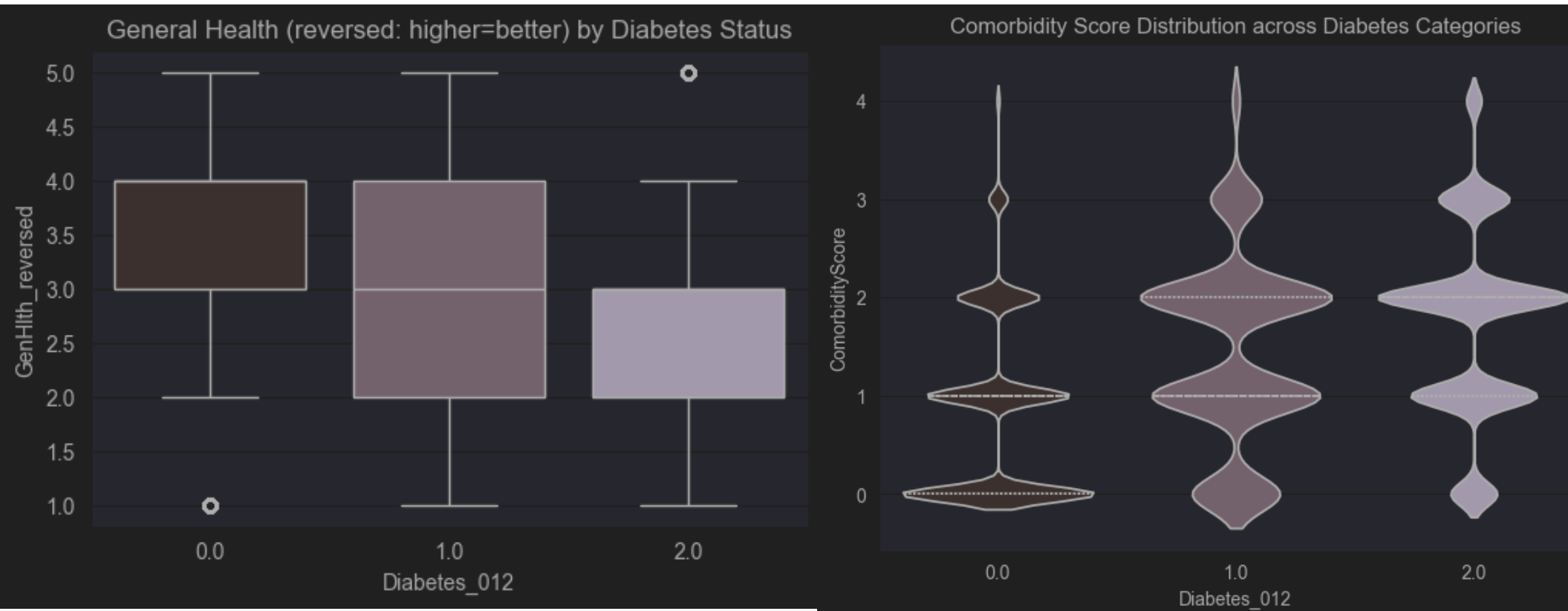


Non diabetic cases:84.24%

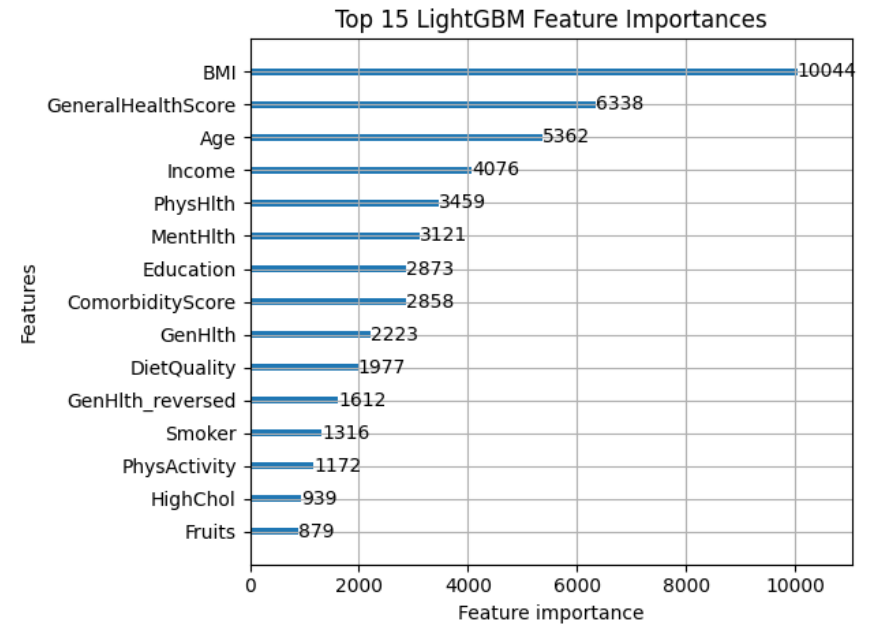
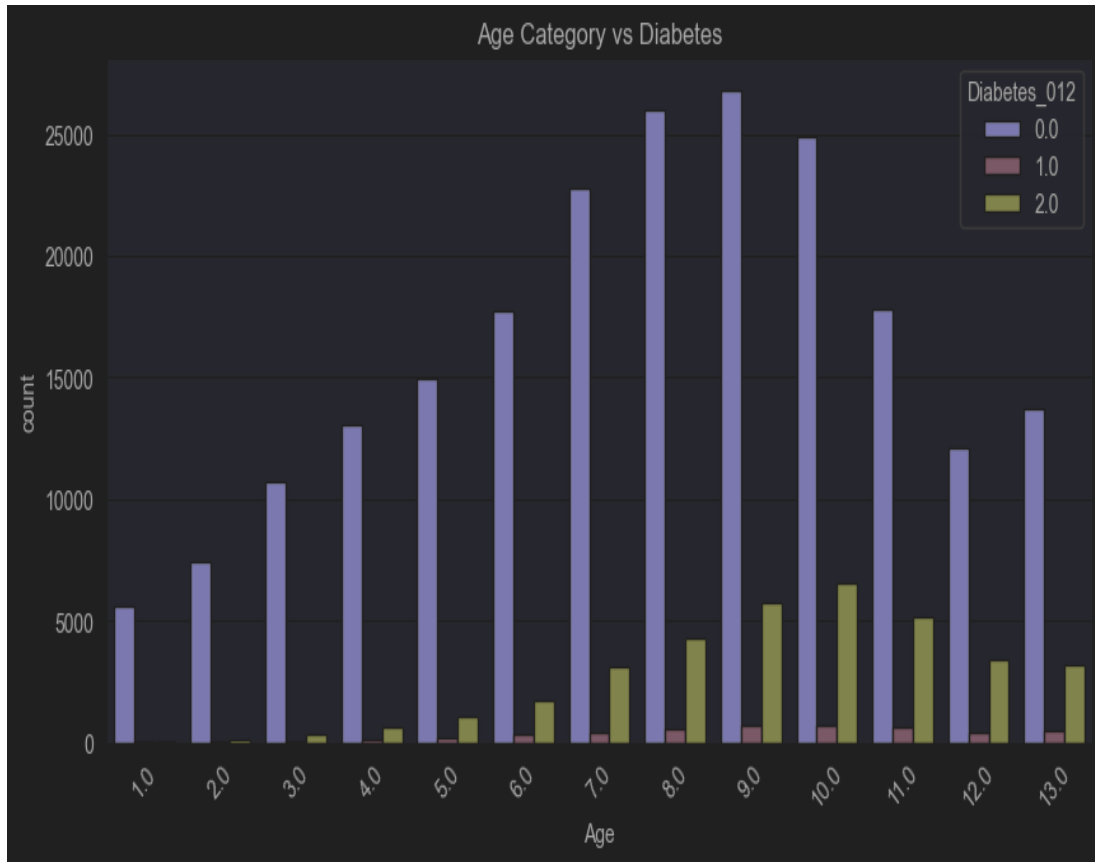
Pre Diabetic cases: 1.82%

Diabetic cases : 13.93%

Data collection and Preprocessing



Data Collection



Machine learning results

```
def engineer_features(df):  
    df = df.copy()  
    df["BMI_SQ"] = df["BMI"] ** 2  
    df["AgeRatio"] = df["Age"] / (df["BMI"] + 1)  
    df["PhysDietCombo"] = df["PhysActivity"] * df["Fruits"]  
    df["BP_Chol"] = df["HighBP"] + df["HighChol"]  
    return df
```

```
X = engineer_features(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.20, random_state=42, stratify=y  
)
```

Important transformations
that help accuracy

Class Voting

- Xgboosting ,SMOTEEN

```
Class 0 - Precision: 0.959, Recall: 0.619, F1: 0.752  
Class 1 - Precision: 0.297, Recall: 0.859, F1: 0.441
```

- Random forest,smote

0.0	0.87	0.95	0.91	42741
1.0	0.01	0.00	0.00	926

- LGBM,SMOTEEN

0.0	0.920	0.785	0.847	145010
1.0	0.362	0.642	0.463	27588

- SVC, Hyprid

0.0	0.96	0.62	0.76	42741
1.0	0.03	0.34	0.05	926

Machine learning results

The Light Gradient Boosting Machine showed the most promising results therefor was chosen

Fine - tuning

```
def objective(trial):
    params = {
        "objective": "binary",
        "metric": "auc",
        "boosting_type": "gbdt",
        "num_leaves": trial.suggest_int("num_leaves", 20, 300),
        "learning_rate": trial.suggest_float("learning_rate", 0.005, 0.2),
        "feature_fraction": trial.suggest_float("feature_fraction", 0.5, 1.0),
        "bagging_fraction": trial.suggest_float("bagging_fraction", 0.5, 1.0),
        "bagging_freq": trial.suggest_int("bagging_freq", 1, 10),
        "max_depth": trial.suggest_int("max_depth", -1, 20),
        "min_data_in_leaf": trial.suggest_int("min_data_in_leaf", 20, 200),
        "lambda_l1": trial.suggest_float("lambda_l1", 0.0, 10.0),
        "lambda_l2": trial.suggest_float("lambda_l2", 0.0, 10.0),
        "min_split_gain": trial.suggest_float("min_split_gain", 0.0, 2.0),
        "scale_pos_weight": trial.suggest_float("scale_pos_weight", 1, 10) # imbalance
    }

    # Stratified 5-fold CV
    kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    aucs = []

    for train_idx, valid_idx in kf.split(X_train, y_train):
        X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[valid_idx]
        y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[valid_idx]

        model = lgb.LGBMClassifier(**params, n_estimators=1500)
        model.fit(
            X_tr, y_tr,
            eval_set=[(X_val, y_val)],
            eval_metric="auc",
            callbacks=[
                lgb.early_stopping(stopping_rounds=100, verbose=False)
            ]
        )
```

Hyper parameter search

```
best_model = lgb.LGBMClassifier(
    **best_params,
    objective="binary",
    metric="auc",
    n_estimators=2000
)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", CalibratedClassifierCV(best_model, method="sigmoid", cv=5))
])

print("\nTraining final model...")
pipe.fit(X_train, y_train)

[12]
```

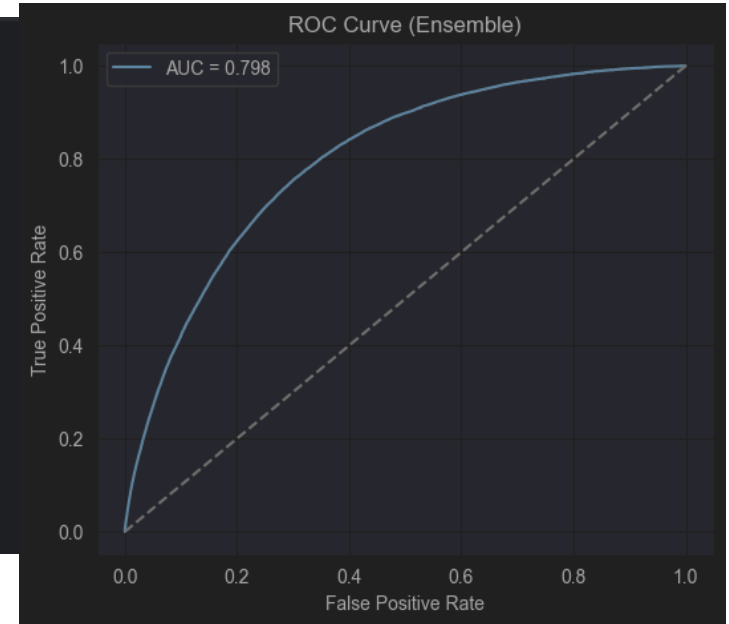
Evaluation

Classification Report:

	precision	recall	f1-score	support
0.0	0.81	0.66	0.72	7070
1.0	0.71	0.84	0.77	7069
accuracy			0.75	14139
macro avg	0.76	0.75	0.75	14139
weighted avg	0.76	0.75	0.75	14139

Confusion matrix

76292	40007
18607	97691



Deployment via the Laravel framework



Documentation



Laravel has wonderful documentation covering every aspect of the framework. Whether you are a newcomer or have prior experience with Laravel, we recommend reading our documentation from beginning to end.



Laracasts



Laracasts offers thousands of video tutorials on Laravel, PHP, and JavaScript development. Check them out, see for yourself, and massively level up your development skills in the process.



Laravel News



Laravel News is a community driven portal and newsletter aggregating all of the latest and most important news in the Laravel ecosystem, including new package releases and tutorials.



Vibrant Ecosystem

Laravel's robust library of first-party tools and libraries, such as [Forge](#), [Vapor](#), [Nova](#), and [Envoyer](#) help you take your projects to the next level. Pair them with powerful open source libraries like [Cashier](#), [Dusk](#), [Echo](#), [Horizon](#), [Sanctum](#), [Telescope](#), and more.

Laravel v10.49.1 (PHP v8.4.11)

deployment

Diabetes Type 2 Risk Prediction

High Blood Pressure (doctor diagnosed)?

Yes

High Cholesterol?

Yes

Cholesterol checked last 5 years?

Yes

BMI

21

Smoker?

Yes

Ever had a stroke?

Yes

Heart disease or attack?

Yes

Exercised in last 30 days?

Yes

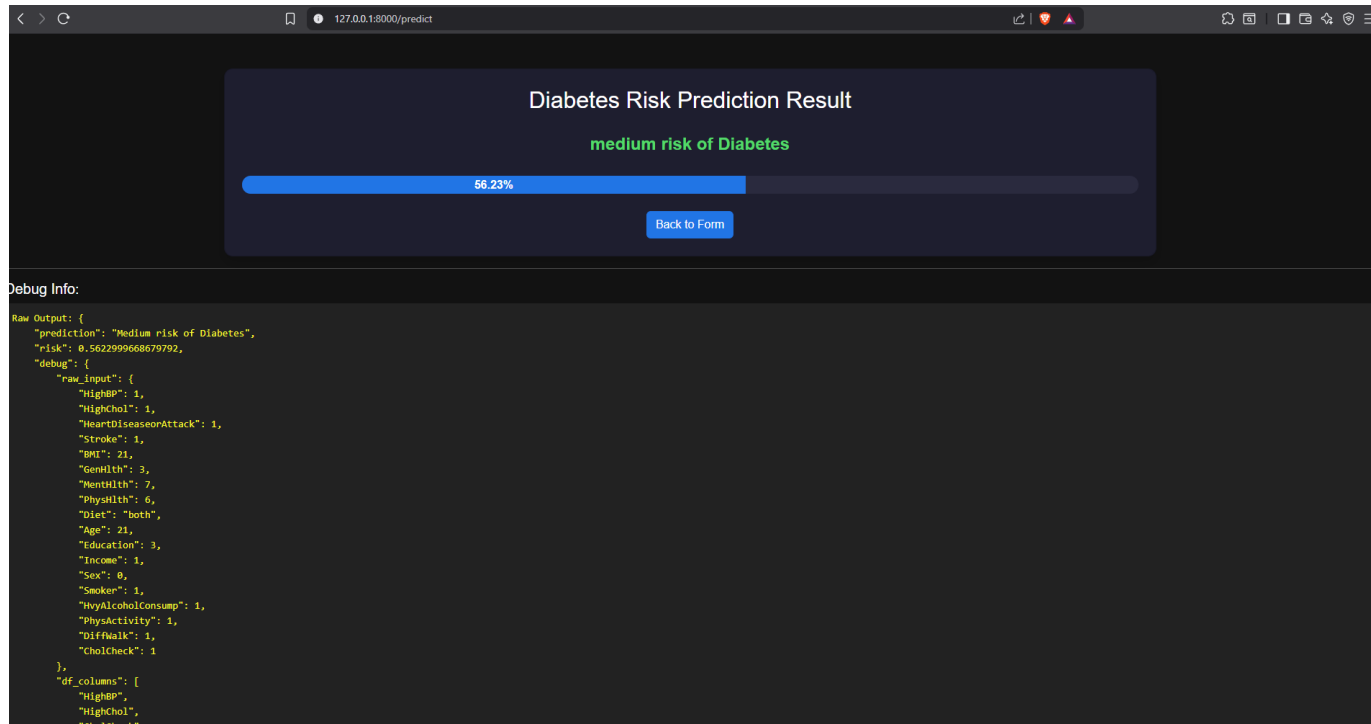
Diet habits

☐ None ☐ Fruits ☐ Veggies ☒ Both

Heavy alcohol consumption?

Yes

Deployment



Future plans

- Adding timeseries system where the model logs the inputs over time for enhanced usage

Thank you !

- Git Hub Repositories
- Main Branch
- Laravel Front end